

平时作业一

2312331 王一诺

一、浮点数有没有移位操作和扩展操作？为什么？

严格来说，浮点数没有像整数那样直接对“整个二进制位串”进行算术移位或逻辑移位的常规运算。整数移位的对象是一个定长二进制补码，左移通常相当于乘以 2，右移通常相当于除以 2 并取整。但浮点数的二进制编码不是一个普通整数，而是由符号位、阶码和尾数组成：

$$(-1)^{\text{sign}} * \text{significand} * 2^{\text{exponent}}$$

如果直接把浮点数的所有位当成整数去左移或右移，符号位、阶码和尾数的边界都会被破坏，结果通常不再对应原来的数值意义。因此，硬件和指令集一般不会把浮点数的“位移”定义成整数移位那样的操作。

不过，浮点数在数值意义上有类似“移位”的操作。因为浮点数本来就是尾数乘以 2 的指数次幂，所以将一个浮点数乘以 2 或除以 2，通常可以理解为调整阶码；当结果进入非规格化数、溢出、下溢或需要舍入时，还需要对尾数进行规格化和舍入处理。也就是说，浮点数可以做“按 2 的幂缩放”的数值操作，但这不是简单的位移。

浮点数有扩展操作。常见的是从低精度浮点格式转换到高精度浮点格式，例如 `float` 转 `double`。这种扩展不是符号扩展或零扩展那种整数扩展，而是浮点格式转换：保留符号，重新编码阶码，并把尾数放到更宽的尾数字段中。

从 `float` 转成 `double` 时，原来 `float` 中能表示的值通常可以被 `double` 精确表示，因为 `double` 的阶码范围和尾数精度都更大。但从 `double` 转成 `float` 时就不一定精确，可能发生舍入、溢出或下溢。

总结来说：

- 浮点数没有整数意义上的普通移位操作，因为浮点编码不是补码整数。
- 浮点数可以通过改变阶码实现乘以或除以 2 的幂，但这属于浮点数值运算。
- 浮点数有精度扩展操作，例如 `float` 到 `double`，本质是浮点格式转换，而不是简单补位。

二、为什么整数除 0 会发生异常而浮点数除 0 不会？

整数除 0 会发生异常，根本原因是整数体系中没有可以表示“无穷大”的普通整数值。对于整数除法：

```
int x = 1 / 0;
```

数学上这个结果没有定义；在机器整数中，也没有一个合法的整数编码可以表示这个结果。处理器执行整数除法指令时，如果除数为 0，通常会触发除法错误异常。例如 x86 上整数 `div` 或 `idiv` 的除数为 0 会触发 Divide Error 异常。

在 C 语言层面，整数除 0 是未定义行为。也就是说，程序语言标准并不要求它一定表现为某一种固定结果，实际运行时通常会由硬件异常、操作系统信号或运行时错误表现出来。

浮点数除 0 的情况不同。现代浮点数通常遵循 IEEE 754 标准。IEEE 754 为一些特殊结果定义了专门的编码，例如：

- `+Infinity`
- `-Infinity`

- NaN
- +0
- -0

因此，对于浮点数：

```
double a = 1.0 / 0.0;    // +Infinity
double b = -1.0 / 0.0;   // -Infinity
double c = 0.0 / 0.0;    // NaN
```

这些情况虽然是异常事件，但不一定中断程序执行。IEEE 754 更常见的处理方式是产生一个特殊结果，并设置浮点异常标志。例如非零数除以 0 会产生带符号的无穷大，同时设置 divide-by-zero 标志；`0.0 / 0.0` 没有确定的数学结果，会产生 NaN，并设置 invalid 标志。

所以，“浮点数除 0 不会异常”这句话要更准确地理解为：默认情况下，浮点除 0 通常不会像整数除 0 那样立即触发同步硬件异常并终止程序，而是返回 IEEE 754 定义的特殊值，并记录异常标志。某些环境也可以开启浮点异常陷阱，开启后浮点除 0 同样可能中断程序。

根本区别在于：

- 整数类型没有无穷大和 NaN 的表示，除 0 没有可返回的整数结果。
- 浮点类型定义了无穷大、NaN 和异常标志，可以继续传播计算结果。

三、为什么同一个实数赋值给 float 型变量和 double 型变量，输出结果会有所不同？

因为 float 和 double 的精度不同。同一个十进制实数写在程序里，最终要被转换成二进制浮点数保存。很多十进制小数无法用有限位二进制精确表示，例如 0.1、0.2、0.3 等。这些数存入浮点变量时，只能存成最接近的可表示二进制浮点数。

float 通常是 IEEE 754 单精度格式，共 32 位：

- 1 位符号位
- 8 位阶码
- 23 位显式尾数
- 有效精度大约是 24 个二进制有效位，约等于 6 到 7 位十进制有效数字

double 通常是 IEEE 754 双精度格式，共 64 位：

- 1 位符号位
- 11 位阶码
- 52 位显式尾数
- 有效精度大约是 53 个二进制有效位，约等于 15 到 16 位十进制有效数字

因此，同一个实数赋值给 float 和 double 时，二者保存的近似值可能不同。例如：

```
float a = 0.1f;
double b = 0.1;
```

a 保存的是最接近 0.1 的单精度浮点数，b 保存的是最接近 0.1 的双精度浮点数。由于 double 的尾数更长，它能保存更接近真实 0.1 的近似值。

输出结果还和打印格式有关。例如：

```
printf("%.20f\n", a);
printf("%.201f\n", b);
```

如果打印很多位小数，就会把二进制近似误差暴露出来；如果只打印较少位数，可能看起来完全一样。这不是 `printf` 随机输出，而是浮点数本身保存的二进制近似值不同，再经过十进制格式化后显示出来。

还要注意，在 C 语言中，`printf` 的可变参数会发生默认实参提升，`float` 传给 `printf` 时会先提升为 `double`。但这个提升不会恢复已经丢失的精度：变量中原来保存的单精度近似值仍然是单精度近似值，只是被精确转换成了一个 `double` 再输出。

所以，同一个实数赋值给 `float` 和 `double` 后输出不同，根本原因是：

- 十进制实数通常不能被二进制浮点数精确表示。
- `float` 和 `double` 的尾数位数不同，舍入后的近似值不同。
- 输出位数越多，越容易看到这种近似误差。

四、为什么 `float` 情况下输出的结果会比原来的大？这是根本原因还是随机发生的？

这不是随机发生的，而是浮点舍入的确定结果。

一个十进制实数赋值给 `float` 时，机器必须在有限个可表示的单精度浮点数中选一个最接近它的值。由于 `float` 的可表示数是离散的，而实数是连续的，所以绝大多数实数都不能被 `float` 精确表示，只能被舍入到某个相邻的可表示值。

假设某个真实十进制数 `x` 位于两个相邻的 `float` 可表示数之间：

```
lower_float < x < upper_float
```

如果 `x` 更接近 `upper_float`，默认的“舍入到最近”规则就会把它保存成 `upper_float`。这时再打印出来，结果就会比原来的十进制数稍大。

如果 `x` 更接近 `lower_float`，保存结果就会比原来的数稍小。如果 `x` 正好在两个可表示数的中间，IEEE 754 默认通常采用“舍入到最近，若正好居中则舍入到尾数最低位为偶数的那个数”，也就是 `round to nearest, ties to even`。

因此，`float` 输出结果比原来的数大，不是随机现象，而是由以下因素确定的：

- 原十进制数在二进制中是否能精确表示。
- 它落在两个相邻 `float` 可表示数之间的位置。
- 当前浮点舍入模式，通常是舍入到最近。
- 输出时保留的小数位数。

例如，很多十进制小数在二进制中会变成无限循环小数：

```
0.1(10) = 0.000110011001100110011... (2)
```

`float` 只能保存有限位尾数，所以必须截断并舍入。舍入后的值可能略大于原来的十进制值，也可能略小于原来的十进制值。

这背后的根本原因是：有限位二进制浮点数无法精确表示所有十进制实数。`float` 的有效位数较少，所以相邻可表示数之间的间隔比 `double` 更大，舍入误差也更容易在输出时被看出来。

因此，正确理解应该是：

- `float` 输出比原数大不是偶然，也不是运行时随机误差。
- 它是二进制浮点表示和舍入规则共同决定的结果。
- 对另一些数，`float` 输出可能比原数小；对少数能精确表示的数，例如 `0.5`、`0.25`、`1.75`，输出可以和原数完全一致。

总结

浮点数和整数的根本区别在于表示方式不同。整数通常使用补码，移位、扩展、除法异常都围绕整数编码规则展开；浮点数使用符号位、阶码和尾数表示近似实数，并由 IEEE 754 定义无穷大、NaN、舍入和异常标志。

所以，浮点数看似出现“多出来一点”或“少了一点”的输出，并不是机器算错，也不是随机误差，而是有限精度二进制表示实数时必然产生的舍入现象。